



پایان نامه دوره کارشناسی  
مهندسی کامپیوتر - گرایش نرم افزار

عنوان پروژه:

طراحی و پیاده سازی ربات تلگرامی برای یاددهی زبان با استفاده  
از مدل هوش مصنوعی (GPT)

دانشجو:

علی کاظم زاده اردستانی

استاد راهنما:

دکتر زینب ترابی

شهریور 1404

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

## تقدیم به: مادرم؛ اولین پناهگاه ها و آخرین تبعید گاه ها

این پایان نامه را با تمام عشق و سپاس تقدیم می کنم به مادرم، که در تمام مسیر تحصیلم چراغ راه و تکیه گاه صبور و مهربانی بود. هر قطره امید و هر لحظه تلاش من از لطف بی دریغ تو نشأت گرفته است. باشد که مرهون زحمات بی پایانت باشم.

## چکیده

این پایان نامه به طراحی و پیاده سازی یک ربات تلگرامی هوشمند برای آموزش زبان انگلیسی با بهره گیری از قابلیت های مدل های زبانی OpenAI مانند (GPT-3.5 Turbo) می پردازد. هدف اصلی این پروژه، تسهیل فرآیند یادگیری زبان از طریق ایجاد تعامل پویا با کاربران و ارائه تمرینات آموزشی متنوع مکالمه، گرامر، واژگان است. ربات با اتصال به OpenAI API، قادر به تولید سوالات، تحلیل پاسخ ها، تصحیح خودکار اشتباهات و شبیه سازی مکالمه است. همچنین، این ربات می تواند تمرینات روزانه را بر اساس سطح کاربر تولید کند. ساختار پایه ربات با استفاده از کتابخانه python-telegram-bot پیاده سازی شده و برای ذخیره سازی اطلاعات کاربران، پیشرفت آن ها و تمرینات، از پایگاه داده SQLite فایل های استفاده گردیده است. نتایج حاصل از این پروژه نشان دهنده پتانسیل بالای مدل های هوش مصنوعی در ایجاد محیط های یادگیری زبان تعاملی و شخصی سازی شده است.

## کلمات کلیدی

ربات تلگرام، آموزش زبان انگلیسی، هوش مصنوعی، OpenAI، پردازش زبان طبیعی

## فهرست مطالب

|       |                                                          |
|-------|----------------------------------------------------------|
| 1     | فصل 1                                                    |
| 1     | مقدمه                                                    |
| 1-1   | 1-1 مقدمه                                                |
| 1-2   | 1-2 بیان مسئله                                           |
| 1-3   | 1-3 اهمیت و ضرورت پروژه                                  |
| 1-4   | 1-4 اهداف پروژه                                          |
| 1-5   | 1-5 ساختار پایان نامه                                    |
| 4     | فصل 2                                                    |
| 4     | پیشینه تحقیق                                             |
| 2-1   | 2-1 مقدمه                                                |
| 2-2   | 2-2 بررسی ربات‌های تلگرامی آموزش زبان موجود              |
| 2-3   | 2-3 بررسی مدل‌های هوش مصنوعی (OpenAI GPT) در آموزش زبان  |
| 2-4   | 2-4 بررسی ابزارهای مرتبط (python-telegram-bot و SQLite)  |
| 2-5   | 2-5 نتیجه‌گیری                                           |
| 2-5-1 | 2-5-1 تحلیل رویکردهای موجود                              |
| 2-5-2 | 2-5-2 مقایسه رویکرد های موجود                            |
| 9     | فصل 3                                                    |
| 9     | طراحی و پیاده‌سازی سیستم                                 |
| 3-1   | 3-1 مقدمه                                                |
| 3-2   | 3-2 معماری کلی سیستم                                     |
| 3-2-1 | 3-2-1 نمودار اجزاء سیستم (Component Diagram)             |
| 3-2-2 | 3-2-2 تشریح اجزاء                                        |
| 3-3   | 3-3 جریان مکالمه و طراحی حالت‌ها (State Machine)         |
| 3-3-1 | 3-3-1 نمودار جامع جریان حالات                            |
| 3-3-2 | 3-3-2 جزئیات پیاده‌سازی 'ConversationHandler'            |
| 3-3-3 | 3-3-3 مدیریت خطاهای مکالمه                               |
| 3-4   | 3-4 طراحی پایگاه داده                                    |
| 3-4-1 | 3-4-1 ER Diagram کامل                                    |
| 3-4-2 | 3-4-2 نرمال‌سازی و یکپارچگی داده‌ها                      |
| 3-4-3 | 3-4-3 استراتژی های indexing                              |
| 3-5   | 3-5 ساختار پروژه و فایل‌ها                               |
| 3-5-1 | 3-5-1 فایل ها و نقش آن ها                                |
| 3-5-2 | 3-5-2 نمونه کدهای کلیدی و توضیحات                        |
| 3-6   | 3-6 مهندسی پرامپت (Prompt Engineering) و سیستم امتیازدهی |
| 3-6-1 | 3-6-1 طراحی پرامپت                                       |
| 3-6-2 | 3-6-2 مکانیزم امتیازدهی                                  |
| 3-7   | 3-7 مدیریت خطا، لاگینگ و امنیت                           |
| 3-7-1 | 3-7-1 مدیریت خطا                                         |
| 3-7-2 | 3-7-2 لاگینگ (Logging)                                   |
| 3-7-3 | 3-7-3 امنیت                                              |
| 41    | فصل 4                                                    |

|    |                                                                   |
|----|-------------------------------------------------------------------|
| 41 | نتایج و ارزیابی                                                   |
| 41 | 4-1 مقدمه                                                         |
| 42 | 4-2 خلاصه دستاوردها                                               |
| 43 | 4-3 محدودیت‌ها                                                    |
| 44 | 4-4 پیشنهادات برای پژوهش‌های بعدی                                 |
| 44 | 4-4-1 توسعه پنل مدیریت و داشبورد تحلیلی (Admin Panel & Analytics) |
| 45 | 4-4-2 ایجاد راهنمای جامع راه‌اندازی و استقرار (Deployment Guide)  |
| 45 | 4-4-3 افزودن قابلیت‌های صوتی و چندرسانه‌ای                        |
| 45 | 4-4-4 پیاده‌سازی گیمیفیکیشن و مسیر یادگیری شخصی‌سازی شده          |
| 46 | 4-4-5 مهاجرت به مدل‌های متن‌باز و بهینه‌سازی (Fine-Tuning)        |
| 46 | 4-5 نتیجه‌گیری                                                    |
| 47 | مراجع                                                             |
| 48 | واژه نامه انگلیسی به فارسی                                        |

## فهرست جدول‌ها

جدول 1-2: مقایسه سه رویکرد مختلف در آموزش زبان جدول ..... 8

## فهرست شکل‌ها

- شکل 1-3: نمودار اجزاء اصلی و جریان داده در سیستم ..... 10
- شکل 2-3 : نمودار جامع حالات ربات و جریان مکالمه ..... 14
- شکل 3-3 : نمودار ER کامل پایگاه داده سیستم ..... 17



# فصل 1

## مقدمه

### 1-1 مقدمه

در عصر حاضر، تسلط بر زبان انگلیسی به عنوان زبان بین‌المللی علم، تجارت و ارتباطات، از اهمیت فزاینده‌ای برخوردار است. با پیشرفت‌های چشمگیر در حوزه هوش مصنوعی و دسترسی گسترده به پلتفرم‌های پیام‌رسان نظیر تلگرام، فرصت‌های نوینی برای توسعه ابزارهای آموزشی تعاملی و شخصی‌سازی شده فراهم آمده است. ربات‌های تلگرامی، به دلیل سهولت دسترسی و محبوبیت بالای این پلتفرم، به بستری ایده‌آل برای ارائه خدمات آموزشی، از جمله آموزش زبان انگلیسی، تبدیل شده‌اند. این ربات‌ها قادرند با ارائه محتوای آموزشی پویا و تعامل مستقیم با کاربران، فرآیند یادگیری زبان را متحول سازند.

### 1-2 بیان مسئله

یادگیری مؤثر زبان انگلیسی، به ویژه در مهارت‌های کلیدی مانند مکالمه، گرامر و واژگان، مستلزم تمرین مداوم و دریافت بازخورد فوری و دقیق است. روش‌های سنتی آموزش زبان اغلب با محدودیت‌هایی نظیر عدم دسترسی کافی به معلمان متخصص، هزینه‌های بالا و عدم امکان شخصی‌سازی محتوا بر اساس نیازهای فردی زبان‌آموزان مواجه هستند. بسیاری از اپلیکیشن‌ها و پلتفرم‌های آموزشی موجود نیز، با وجود مزایای خود، فاقد قابلیت‌های پیشرفته برای تعامل هوشمند و تولید محتوای کاملاً شخصی‌سازی شده هستند. این کاستی

ها می‌تواند منجر به کاهش انگیزه و ناکارآمدی در فرآیند یادگیری شود. بنابراین، نیاز به یک راهکار آموزشی نوین که بتواند این چالش‌ها را برطرف کرده و تجربه‌ای جامع، شخصی‌سازی شده و تعاملی را برای زبان‌آموزان فراهم آورد، به شدت احساس می‌شود.

### 3-1 اهمیت و ضرورت پروژه

پروژه حاضر با هدف پاسخگویی به نیازهای فوق و ارائه یک راهکار مبتکرانه برای آموزش زبان انگلیسی طراحی و پیاده‌سازی شده است. با بهره‌گیری از قدرت مدل‌های زبانی بزرگ (LLMs) مانند OpenAI API مانند (GPT-3.5 Turbo)، این ربات تلگرامی قادر است محتوای آموزشی را به صورت پویا و در لحظه تولید کند و آن را با سطح و نیازهای خاص هر کاربر تطبیق دهد. این رویکرد نه تنها فرآیند یادگیری را جذاب‌تر و کارآمدتر می‌سازد، بلکه با فراهم آوردن امکان تمرین مداوم و دریافت بازخورد فوری، به افزایش سرعت و کیفیت یادگیری کمک شایانی می‌کند. اهمیت این پروژه در توانایی آن برای دموکراتیزه کردن دسترسی به آموزش زبان با کیفیت بالا، از طریق یک پلتفرم در دسترس و کاربرپسند مانند تلگرام، نهفته است [1].

### 4-1 اهداف پروژه

اهداف اصلی این پروژه عبارتند از:

- طراحی و پیاده‌سازی یک ربات تلگرامی تعاملی با استفاده از کتابخانه python-telegram-bot برای آموزش زبان انگلیسی.
- یکپارچه‌سازی مدل‌های زبانی OpenAI مانند (GPT-3.5 Turbo) برای تولید محتوای آموزشی پویا، تحلیل پاسخ‌های کاربران و شبیه‌سازی مکالمه.
- توسعه ماژول‌هایی برای ارائه تمرینات متنوع در حوزه‌های واژگان، گرامر و مکالمه.
- پیاده‌سازی قابلیت تصحیح خودکار پاسخ‌های کاربران و ارائه بازخورد دقیق و فوری.

- طراحی و پیاده‌سازی سیستمی برای سنجش سطح زبان کاربران و ارائه محتوای متناسب با آن.
- ایجاد قابلیت تولید تمرینات روزانه و پیگیری پیشرفت کاربران.
- طراحی و استفاده از پایگاه داده (SQLite) فایل‌های (user\_data.db) و (content\_data.db) برای مدیریت اطلاعات کاربران، سوابق آموزشی و محتوای ربات.

## 5-1 ساختار پایان نامه

این پایان‌نامه در پنج فصل سازماندهی شده است. فصل اول به معرفی کلی پروژه، بیان مسئله، اهمیت و ضرورت، اهداف و ساختار پایان‌نامه می‌پردازد. در فصل دوم، پیشینه تحقیق شامل بررسی ربات‌های تلگرامی آموزش زبان موجود، مدل‌های هوش مصنوعی (به ویژه GPT و ابزارهای مرتبط مانند SQLite و python-telegram-bot) مورد بحث قرار می‌گیرد. فصل سوم به روش تحقیق، شامل معماری سیستم، طراحی پایگاه داده و جزئیات پیاده‌سازی ماژول‌های مختلف اختصاص دارد. در فصل چهارم، نتایج حاصل از پیاده‌سازی و ارزیابی عملکرد ربات ارائه و مورد بحث قرار می‌گیرد. در نهایت، فصل پنجم شامل نتیجه‌گیری کلی، دستاوردهای پروژه و پیشنهادات برای کارهای آتی خواهد بود.

## فصل 2

### پیشینه تحقیق

#### 2-1 مقدمه

در این فصل، به بررسی تحقیقات و پروژه‌های مرتبط با آموزش زبان انگلیسی و کاربرد هوش مصنوعی در این حوزه می‌پردازیم. هدف از این بررسی، شناسایی نقاط قوت و ضعف رویکردهای موجود، درک شکاف‌های تحقیقاتی و تعیین جایگاه پروژه حاضر در میان کارهای پیشین است. ابتدا به مرور ربات‌های تلگرامی موجود برای آموزش زبان می‌پردازیم، سپس کاربرد مدل‌های هوش مصنوعی، به ویژه مدل‌های OpenAI GPT، در آموزش زبان را بررسی می‌کنیم و در نهایت، ابزارهای مرتبط با پیاده‌سازی ربات‌های تلگرامی مانند python-telegram-bot و پایگاه داده SQLite را مورد تحلیل قرار می‌دهیم.

#### 2-2 بررسی ربات‌های تلگرامی آموزش زبان موجود

. با گسترش پلتفرم تلگرام، ربات‌های متعددی برای اهداف آموزشی، از جمله آموزش زبان انگلیسی، توسعه یافته‌اند. این ربات‌ها معمولاً امکاناتی نظیر ارائه واژگان جدید، تمرینات گرامر، جمله‌سازی و حتی مکالمه‌های ساده را فراهم می‌کنند. برخی از این ربات‌ها از پایگاه داده‌های ثابت برای ارائه محتوا استفاده می‌کنند، در حالی که برخی دیگر ممکن است از API های خارجی برای ترجمه یا دیکشنری بهره ببرند [2]. این حال، بسیاری از این ربات‌ها دارای محدودیت‌هایی هستند:

- **عدم شخصی‌سازی کافی:** محتوای آموزشی اغلب برای همه کاربران یکسان است و کمتر به سطح دانش یا نیازهای خاص هر فرد توجه می‌شود.

- **تعامل محدود:** مکالمات معمولاً بر اساس الگوهای از پیش تعریف شده صورت می‌گیرد و فاقد انعطاف پذیری و هوشمندی لازم برای یک مکالمه طبیعی است.
  - **بازخورد غیردقیق:** تصحیح اشتباهات کاربران ممکن است محدود به پاسخ‌های از پیش تعیین شده باشد و نتواند خطاهای پیچیده‌تر را شناسایی و توضیح دهد.
  - **محتوای ثابت:** محتوای آموزشی به روز نمی‌شود و یا تنوع کافی ندارد، که می‌تواند منجر به خستگی کاربر شود.
- به عنوان مثال، ربات‌هایی مانند 'English Speaking Bot' یا 'Grammar Test Bot' در تلگرام وجود دارند که هر کدام بر جنبه خاصی از آموزش زبان تمرکز دارند، اما کمتر رباتی یافت می‌شود که بتواند یک تجربه جامع و کاملاً شخصی‌سازی شده را ارائه دهد. این ربات‌ها معمولاً به عنوان ابزارهای مکمل عمل می‌کنند و نمی‌توانند جایگزین یک معلم یا یک پلتفرم آموزشی هوشمند شوند.

### 3-2 بررسی مدل‌های هوش مصنوعی (OpenAI GPT) در آموزش زبان

مدل‌های زبانی بزرگ (LLMs) مانند GPT (Generative Pre-trained Transformer) که توسط OpenAI توسعه یافته‌اند، انقلابی در حوزه پردازش زبان طبیعی (NLP) ایجاد کرده‌اند. این مدل‌ها با آموزش بر روی حجم عظیمی از داده‌های متنی، قادر به تولید متن‌های منسجم و مرتبط، پاسخگویی به سوالات، خلاصه‌سازی، ترجمه و حتی شبیه‌سازی مکالمه انسانی هستند. قابلیت‌های این مدل‌ها، آنها را به ابزاری قدرتمند برای کاربردهای آموزشی، به ویژه در زمینه آموزش زبان، تبدیل کرده است. در این پروژه، از مدل‌های OpenAI API (مانند GPT-3.5 Turbo) استفاده شده است که در فایل bot.py به وضوح مشخص است.

کاربردهای OpenAI GPT در آموزش زبان شامل موارد زیر است:

- تولید محتوای آموزشی: OpenAI GPT می‌تواند تمرینات گرامر، واژگان، جمله‌سازی و حتی داستان‌های کوتاه را بر اساس سطح و نیاز کاربر تولید کند. این قابلیت امکان شخصی‌سازی بینظیری را فراهم می‌آورد.
- شبیه‌سازی مکالمه: با توجه به توانایی OpenAI GPT در تولید پاسخ‌های طبیعی و مرتبط، میتوان از آن برای شبیه‌سازی مکالمات با زبان‌آموزان استفاده کرد. این امر به کاربران اجازه می‌دهد تا مهارت‌های مکالمه خود را در یک محیط امن و بدون استرس تمرین کنند.

- تصحیح و بازخورد: OpenAI GPT میتواند اشتباهات گرامری، املائی و حتی ساختاری را در نوشته‌ها یا گفتار کاربران شناسایی کرده و بازخورد دقیق و توضیحات لازم را ارائه دهد. این بازخورد میتواند به صورت فوری و شخصی‌سازی شده باشد.
- توضیح مفاهیم پیچیده: مدل‌های OpenAI GPT میتوانند مفاهیم گرامری یا واژگان دشوار را به زبان ساده و با مثال‌های متنوع توضیح دهند، که این امر به درک عمیق‌تر مطالب کمک میکند.
- با این حال، استفاده از OpenAI GPT در آموزش زبان نیز چالش‌هایی دارد، از جمله نیاز به کنترل خروجی برای اطمینان از صحت اطلاعات، مدیریت تعاملات طولانی‌مدت و حفظ حریم خصوصی کاربران. با وجود این چالش‌ها، پتانسیل OpenAI GPT در متحول کردن آموزش زبان بسیار زیاد است و پروژه‌های تحقیقاتی متعددی در این زمینه در حال انجام است.[3]

## 4-2 بررسی ابزارهای مرتبط (python-telegram-bot و SQLite)

برای پیاده‌سازی ربات‌های تلگرامی، کتابخانه‌ها و فریم‌ورک‌های مختلفی در زبان‌های برنامه‌نویسی گوناگون وجود دارد. در میان این ابزارها، python-telegram-bot یکی از محبوب‌ترین و قدرتمندترین کتابخانه‌ها برای توسعه ربات‌های تلگرامی با زبان پایتون است. این کتابخانه، یک رابط برنامه‌نویسی کاربردی (API) سطح بالا را برای تعامل با Telegram Bot API فراهم می‌کند و امکاناتی نظیر مدیریت به‌روزرسانی‌ها، ارسال پیام‌ها، مدیریت دستورات و پاسخ به رویدادهای مختلف را به سادگی فراهم می‌آورد. ویژگی‌های کلیدی python-telegram-bot که آن را برای این پروژه مناسب می‌سازد، عبارتند از:

- **سادگی و سهولت استفاده:** این کتابخانه دارای مستندات جامع و مثال‌های فراوان است که توسعه ربات را برای برنامه‌نویسان آسان می‌کند.
- **مدیریت به‌روزرسانی‌ها:** python-telegram-bot به توسعه‌دهندگان اجازه می‌دهد تا به‌روزرسانی‌های دریافتی از تلگرام را به صورت کارآمد مدیریت کنند، چه از طریق پولینگ (Polling) و چه از طریق وب‌هوک (Webhook).
- **پشتیبانی از انواع پیام‌ها:** این کتابخانه از انواع مختلف پیام‌ها شامل متن، عکس، ویدئو، فایل و ... پشتیبانی می‌کند.

- **قابلیت‌های پیشرفته:** امکاناتی مانند کیبوردهای سفارشی، دکمه‌های درون خطی، و مدیریت وضعیت کاربر (ConversationHandler) برای ساخت ربات‌های پیچیده و تعاملی فراهم شده است.

علاوه بر python-telegram-bot، برای ذخیره‌سازی اطلاعات کاربران و داده‌های آموزشی، از پایگاه داده SQLite استفاده شده است. SQLite یک سیستم مدیریت پایگاه داده رابطه‌ای (RDBMS) سبک‌وزن و بدون سرور است که داده‌ها را در یک فایل واحد ذخیره می‌کند. این ویژگی آن را برای پروژه‌های کوچک و متوسط، و همچنین برای توسعه و آزمایش، بسیار مناسب می‌سازد. در این پروژه، دو فایل پایگاه داده user\_data.db و content\_data.db برای مدیریت اطلاعات کاربران و محتوای آموزشی به ترتیب استفاده شده اند. فایل user\_db.py مسئول مدیریت تعاملات با user\_data.db و فایل content\_manager.py مسئول مدیریت تعاملات با content\_data.db است [4].

## 5-2 نتیجه گیری

با توجه به بررسی‌های انجام شده در بخش‌های قبلی، می‌توان دریافت که ربات‌های تلگرامی موجود برای آموزش زبان، هرچند مفید هستند، اما در زمینه شخصی‌سازی و تعامل هوشمند با کاربر، دارای محدودیت‌هایی می‌باشند. این ربات‌ها اغلب بر اساس قواعد از پیش تعریف شده عمل می‌کنند و قادر به درک عمیق زبان طبیعی و تولید پاسخ‌های خلاقانه نیستند. در مقابل، مدل‌های هوش مصنوعی مانند OpenAI GPT، قابلیت‌های بی‌نظیری در تولید زبان طبیعی، درک متن و شبیه‌سازی مکالمه ارائه می‌دهند که می‌تواند این محدودیت‌ها را برطرف کند [5].

## 1-5-2 تحلیل رویکردهای موجود

رویکرد پیشنهادی در این پروژه، ترکیب نقاط قوت ربات‌های تلگرامی (سهولت دسترسی و استفاده) با قدرت مدل‌های هوش مصنوعی (شخصی‌سازی، تعامل هوشمند و تولید محتوای پویا) است. این ترکیب به ما امکان می‌دهد تا یک ربات آموزشی ایجاد کنیم که نه تنها در دسترس و کاربرپسند باشد، بلکه بتواند تجربه‌ای کاملاً

شخصی سازی شده و پویا را برای هر زبان آموز فراهم آورد. استفاده از python-telegram-bot به عنوان بستر پیاده سازی، امکان اتصال آسان به API تلگرام و بهره گیری از قابلیت های این پلتفرم را فراهم می کند، در حالی که اتصال به OpenAI API مدل های (GPT)، هوشمندی و انعطاف پذیری لازم را برای تولید محتوای آموزشی و شبیه سازی مکالمه به ربات می بخشد. استفاده از SQLite نیز مدیریت داده ها را به صورت کارآمد و محلی فراهم می آورد.

## 2-5-2 مقایسه رویکرد های موجود

در جدول 1-2، مقایسه ای بین رویکردهای سنتی، ربات های تلگرامی موجود و رویکرد پیشنهادی در این پروژه ارائه شده است:

جدول 1-2: مقایسه سه رویکرد مختلف در آموزش زبان جدول

| ویژگی / رویکرد | روش های سنتی   | ربات های تلگرامی موجود | رویکرد پیشنهادی |
|----------------|----------------|------------------------|-----------------|
| شخصی سازی      | پایین          | متوسط                  | بالا            |
| تعامل پذیری    | متوسط          | متوسط                  | بالا            |
| تولید محتوا    | ثابت           | ثابت/محدود             | پویا و نامحدود  |
| بازخورد        | با تاخیر/محدود | محدود                  | فوری و دقیق     |
| مکالمه         | محدود          | الگو-محور              | طبیعی و هوشمند  |
| دسترسی پذیری   | متوسط          | بالا                   | بالا            |
| هوشمندی        | پایین          | پایین                  | بالا            |

این مقایسه نشان می دهد که رویکرد پیشنهادی، با بهره گیری از هوش مصنوعی، می تواند یک گام مهم در جهت بهبود کیفیت و کارایی آموزش زبان انگلیسی باشد [5,6,7].



## فصل ۳

# طراحی و پیاده‌سازی سیستم

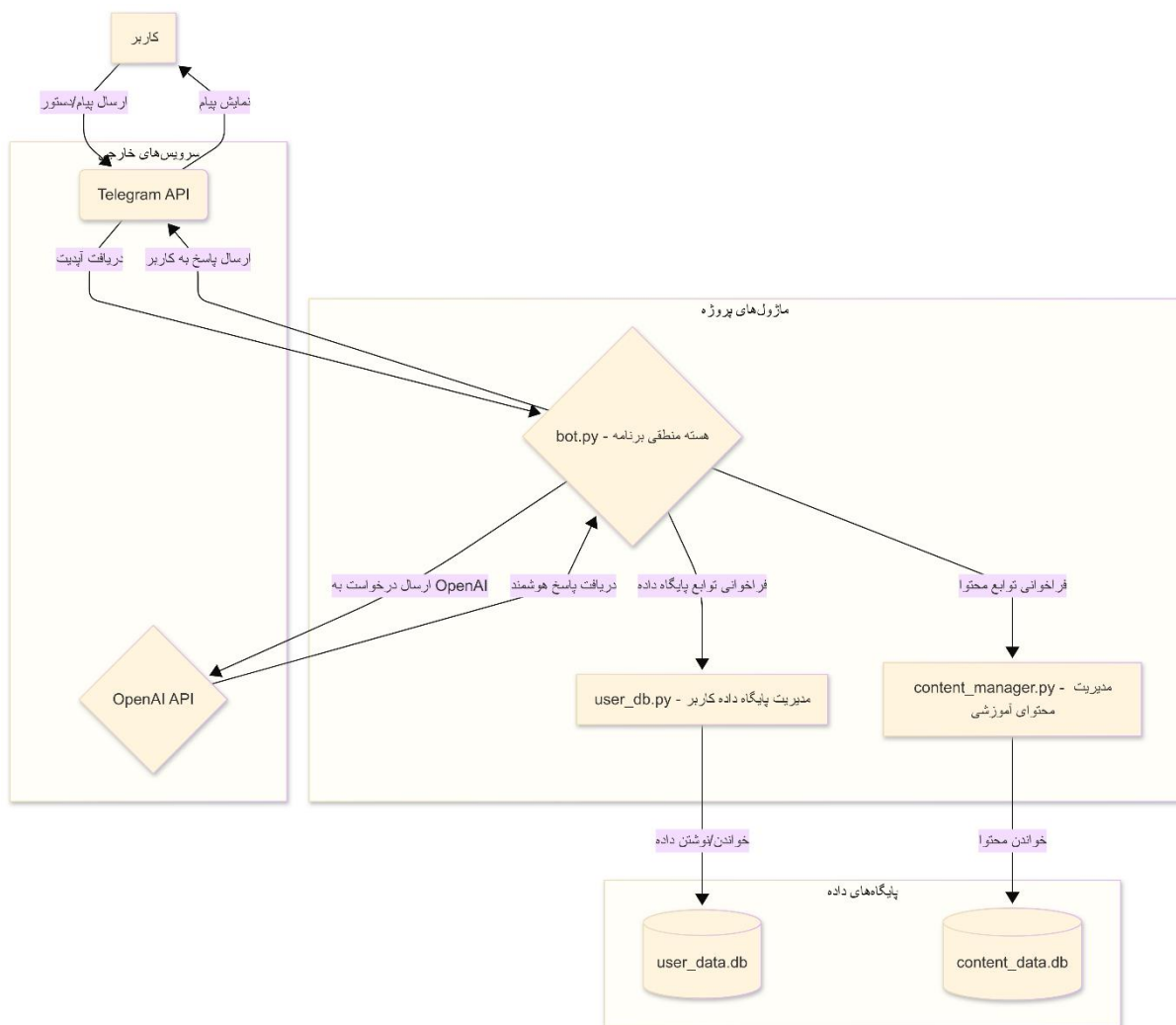
### 3-1 مقدمه

این فصل به تشریح دقیق فرآیند طراحی، معماری و پیاده‌سازی فنی “ربات هوشمند آموزش زبان” می‌پردازد. هدف این فصل، ارائه یک تصویر شفاف از اجزای داخلی سیستم، منطق نرم‌افزاری، ساختار داده‌ها و تکنولوژی‌های به کار رفته است. در این راستا، ابتدا معماری کلی سیستم و نحوه تعامل ماژول‌های مختلف آن شرح داده می‌شود. سپس، جریان‌های کاری اصلی کاربر، از جمله فرآیند ثبت‌نام و آزمون تعیین سطح، به تفصیل بررسی می‌گردد. در ادامه، ساختار و اسکیمای پایگاه‌های داده‌ای که وظیفه ذخیره‌سازی اطلاعات کاربران و محتوای آموزشی را بر عهده دارند، نمایش داده خواهد شد. بخش‌های بعدی به بررسی ساختار کد پروژه، تحلیل ماژول‌های کلیدی به همراه نمونه کدهای واقعی و تشریح تکنیک‌های مهندسی پرامپت برای تعامل با مدل زبانی OpenAI اختصاص دارد. در نهایت، ملاحظات امنیتی، مدیریت خطا و سیستم لاگینگ پروژه مورد بحث قرار می‌گیرد.

## 3-2 معماری کلی سیستم

سیستم بر اساس یک معماری ماژولار و لایه‌ای طراحی شده است تا ضمن تفکیک مسئولیت‌ها (Separation of Concerns)، قابلیت نگهداری، تست و توسعه‌پذیری آن در آینده افزایش یابد. این معماری از چندین جزء اصلی تشکیل شده که به صورت هماهنگ با یکدیگر کار می‌کنند.

### 3-2-1 نمودار اجزاء سیستم (Component Diagram)



شکل 3-1: نمودار اجزاء اصلی و جریان داده در سیستم

نمودار بالا اجزای اصلی سیستم و جریان داده بین آن‌ها را نمایش می‌دهد. این نمودار یک دید کلی از نحوه تعامل ماژول‌های مختلف، از رابط کاربری تلگرام گرفته تا سرویس هوش مصنوعی، فراهم می‌کند.

## 2-2-3 تشریح اجزاء

1. **رابط کاربری (Telegram API):** این لایه، نقطه ورود تمام تعاملات کاربر است. پیام‌ها، دستورات (/start, /test) و کلیک روی دکمه‌های کیبورد سفارشی از طریق API تلگرام دریافت شده و به هسته برنامه ارسال می‌شوند. پاسخ‌های تولید شده توسط سیستم نیز از همین طریق برای کاربر نمایش داده می‌شوند.

## 2. هسته منطقی برنامه (bot.py):

این فایل، قلب تپنده و مرکز کنترل سیستم است. مسئولیت‌های اصلی این ماژول عبارتند از:

- راه‌اندازی و مقداردهی اولیه: ایجاد نمونه‌های 'Updater' و 'Dispatcher' از کتابخانه 'python-telegram-bot' برای دریافت آپدیت‌ها از تلگرام و هدایت آن‌ها به هندلرهای مربوطه.
- ثبت هندلرها (Handlers): تعریف و ثبت تمام هندلرهای رویداد، شامل 'CommandHandler' برای دستورات، 'MessageHandler' برای پیام‌های متنی و 'CallbackQueryHandler' برای دکمه‌های شیشه‌ای (Inline Buttons). مهم‌ترین بخش در این قسمت، استفاده از 'ConversationHandler' برای مدیریت مکالمات چندمرحله‌ای و پیچیده مانند آزمون تعیین سطح و تمرین‌های تعاملی است. این هندلر به ربات اجازه می‌دهد تا وضعیت (State) مکالمه با هر کاربر را به صورت مجزا دنبال کند.
- هماهنگی بین ماژول‌ها: این ماژول به عنوان یک ارکستراتور عمل کرده و درخواست‌ها را به ماژول‌های دیگر (مانند 'user\_db.py' و 'content\_manager.py') ارسال می‌کند. برای مثال، پس از دریافت دستور '/start'، این ماژول تابع 'register\_user' را از 'user\_db.py' فراخوانی می‌کند.
- فراخوانی سرویس هوش مصنوعی: برای وظایف پیچیده‌ای مانند ارزیابی پاسخ کاربر در تمرین لغات یا پیشبرد مکالمه آزاد، این ماژول با 'OpenAI API' ارتباط برقرار کرده، پرامپت مناسب را ارسال و پاسخ دریافت شده را پردازش می‌کند.

3. ماژول مدیریت پایگاه داده کاربر (**user\_db.py**) این ماژول به عنوان یک لایه انتزاعی

(Abstraction Layer) برای پایگاه داده user\_data.db عمل می‌کند و تمام عملیات CRUD (Create, Read, Update, Delete) مربوط به کاربران، پیشرفت آن‌ها و سوابق تمرین‌ها را کپسوله می‌کند.

4. ماژول مدیریت محتوا (**content\_manager.py**) این بخش مسئولیت دسترسی به پایگاه داده ایستا

content\_data.db را دارد. وظیفه آن، واکنشی محتوای آموزشی مانند سوالات آزمون تعیین سطح، دروس گرامر و لیست واژگان بر اساس سطح کاربر است.

5. سرویس هوش مصنوعی (**OpenAI API**)

این جزء خارجی، مغز متفکر سیستم برای وظایف زبانی پیچیده است. با استفاده از مدل `GPT-3.5-turbo`، این سرویس قابلیت‌های زیر را برای ربات فراهم می‌کند:

- ارزیابی هوشمند: ارزیابی جملات ساخته شده توسط کاربر در تمرینات لغات و گرامر بر اساس معیارهای چندگانه (صحت گرامری، استفاده صحیح از کلمه، طبیعی بودن جمله).
- تولید بازخورد سازنده: ارائه بازخوردهای دقیق و تشویق‌کننده به کاربر برای کمک به یادگیری عمیق‌تر.
- مکالمه آزاد: پیشبرد مکالمات طبیعی و پویا در بخش "Chat with AI" برای تقویت مهارت گفتگوی کاربر.

6. پایگاه‌های داده (user\_data.db & content\_data.db)

سیستم از دو پایگاه داده مجزای SQLite استفاده می‌کند:

- `user_data.db`` (پویا): این پایگاه داده تمام اطلاعات مربوط به کاربران که در طول زمان تغییر می‌کند را ذخیره می‌نماید؛ از جمله اطلاعات پروفایل، سطح، پیشرفت در بخش‌های مختلف و سوابق تمرین‌ها.

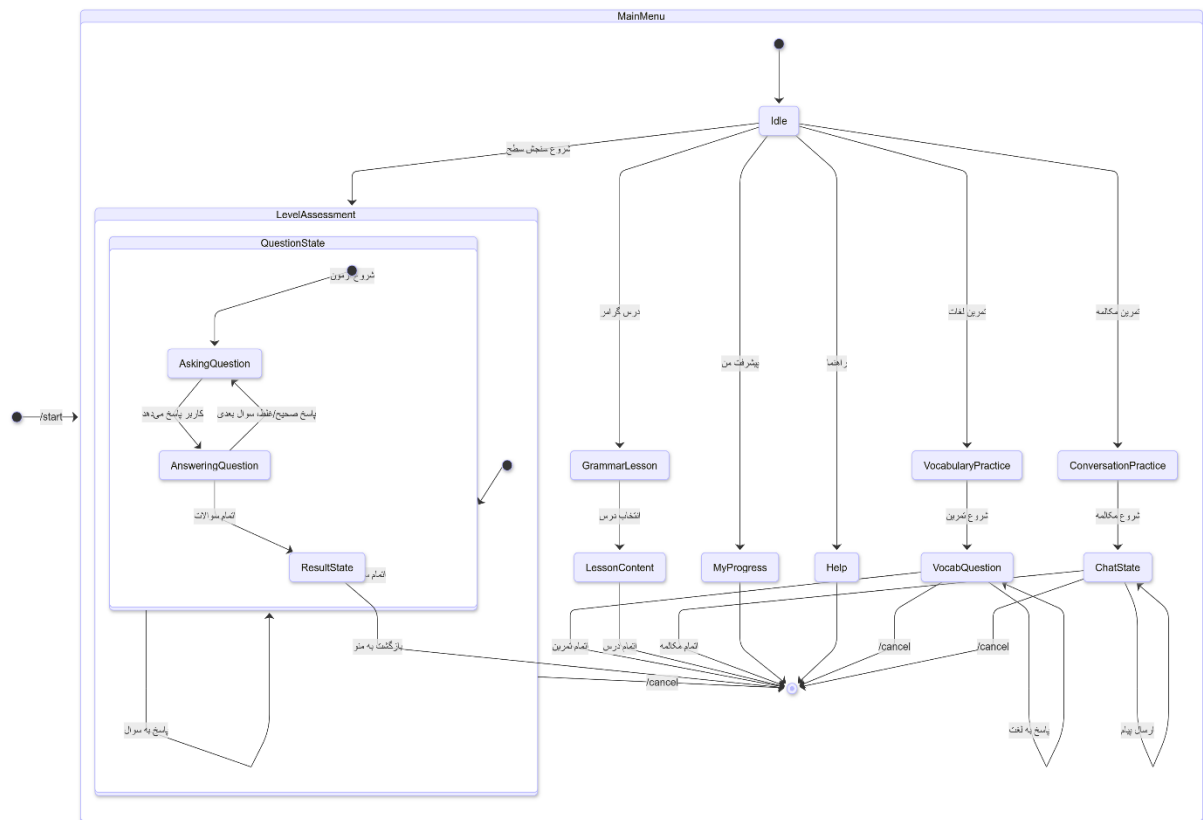
- `content_data.db`` (ایستا): این پایگاه داده حاوی محتوای آموزشی است که به ندرت تغییر می‌کند؛ مانند سوالات آزمون، دروس گرامر و لیست لغات. تفکیک این دو پایگاه داده، مدیریت، پشتیبان‌گیری و بهینه‌سازی عملکرد را ساده‌تر می‌کند.

### 3-3 جریان مکالمه و طراحی حالت‌ها (State Machine)

برای مدیریت فرآیندهای پیچیده‌تر که نیازمند چندین مرحله تعامل با کاربر هستند، از `ConversationHandler` کتابخانه `python-telegram-bot` استفاده شده است که یک پیاده‌سازی عملی از الگوی ماشین حالت (State Machine) است. مهم‌ترین جریان کاری در این سیستم، “آزمون تعیین سطح” است.

#### 3-3-1 نمودار جامع جریان حالات

نمودار زیر، یک دید کلی از تمام حالات اصلی ربات و نحوه انتقال بین آن‌ها را نشان می‌دهد. این نمودار شامل فلوهای اصلی مانند منوی اصلی، آزمون تعیین سطح، تمرین لغات، درس گرامر و مکالمه آزاد است. هر یک از این بخش‌ها می‌توانند خود شامل زیرحالت‌های پیچیده‌تری باشند که در نمودار اصلی به صورت خلاصه نمایش داده شده‌اند.



شکل 2-3: نمودار جامع حالات ربات و جریان مکالمه

### 3-3-2 جزئیات پیاده‌سازی `ConversationHandler`

`ConversationHandler` یکی از قدرتمندترین ابزارهای `python-telegram-bot` برای مدیریت فلوهای مکالمه‌ای است که شامل چندین مرحله می‌شود. این هندلر به ربات اجازه می‌دهد تا وضعیت فعلی مکالمه با هر کاربر را ردیابی کند و بر اساس آن، ورودی‌های کاربر را به هندلرهای مناسب هدایت کند. اجزای اصلی `ConversationHandler` عبارتند از:

- ``entry_points``: لیستی از هندلرها که مکالمه را آغاز می‌کنند. به عنوان مثال، ``CommandHandler('test', start_test)`` مکالمه آزمون تعیین سطح را با دستور ``test/`` آغاز می‌کند.

- `'states'`: یک دیکشنری که وضعیت‌های مختلف مکالمه و هندلرهای مربوط به هر وضعیت را تعریف می‌کند. هر کلید در این دیکشنری یک وضعیت (`State`) است و مقدار آن لیستی از هندلرها (مانند `'MessageHandler'` یا `'CallbackQueryHandler'`) است که در آن وضعیت خاص فعال هستند. به عنوان مثال، در وضعیت `'QUESTION'` برای آزمون تعیین سطح، تنها `'MessageHandler'` که پاسخ‌های متنی کاربر را دریافت می‌کند، فعال است.
- `'fallbacks'`: لیستی از هندلرها که در صورت عدم تطابق ورودی کاربر با هیچ یک از هندلرهای تعریف شده در وضعیت فعلی، یا برای خروج اضطراری از مکالمه (مانند دستور `'cancel/'`) فعال می‌شوند. این هندلرها معمولاً مکالمه را به پایان می‌رسانند یا کاربر را به منوی اصلی بازمی‌گردانند.
- `'map_to_parent'`: این پارامتر به `'ConversationHandler'`‌های تو در تو (`Nested ConversationHandlers`) اجازه می‌دهد تا وضعیت خود را به هندلر والد منتقل کنند. این قابلیت برای ساخت فلوهای پیچیده‌تر و ماژولار بسیار مفید است.

با استفاده از `'ConversationHandler'`، ربات می‌تواند به طور هوشمندانه وضعیت هر کاربر را مدیریت کند. به عنوان مثال، در طول آزمون تعیین سطح، ربات می‌داند که کاربر در حال پاسخ به یک سوال است و ورودی او را به عنوان پاسخ سوال فعلی تفسیر می‌کند، نه یک دستور عمومی یا یک پیام مکالمه آزاد. این رویکرد، پیچیدگی مدیریت فلوهای مکالمه را به شدت کاهش می‌دهد و به توسعه‌دهنده اجازه می‌دهد تا بر منطق کسب‌وکار تمرکز کند.

### 3-3-3 مدیریت خطاهای مکالمه

مدیریت خطا در طول مکالمه برای حفظ تجربه کاربری مثبت و پایداری ربات حیاتی است. ربات هوشمند آموزش زبان، رویکردهای زیر را برای مدیریت خطاهای مکالمه اتخاذ می‌کند:

- اعتبارسنجی ورودی کاربر (`validate_user_input``): قبل از پردازش هر ورودی متنی از کاربر (به ویژه در تمرینات لغات و مکالمه)، تابع `validate_user_input`` فراخوانی می‌شود. این تابع ورودی را از نظر حداقل طول، وجود کاراکترهای نامعتبر، تکرار بیش از حد حروف، و وجود حروف انگلیسی بررسی می‌کند. در صورت نامعتبر بودن ورودی، یک پیام خطای دوستانه به کاربر ارسال می‌شود و از پردازش بیشتر جلوگیری می‌گردد. این کار از ارسال پرامپت‌های بی‌معنی به OpenAI API و هدر رفتن توکن‌ها نیز جلوگیری می‌کند.
- هندلرهای Fallback: همانطور که پیش‌تر ذکر شد، `fallbacks`` در `ConversationHandler`` برای مدیریت شرایطی که ورودی کاربر با هیچ یک از هندلرهای انتظار رفته در وضعیت فعلی مطابقت ندارد، استفاده می‌شوند. این مکانیزم به ربات اجازه می‌دهد تا در صورت سردرگمی کاربر یا تلاش برای خروج از یک فلو، به صورت graceful به وضعیت قبلی بازگردد یا مکالمه را به پایان برساند.
- مدیریت استثنائات در توابع Asynchronous: تمام توابع `async`` که با Telegram API یا OpenAI API تعامل دارند، در بلوک‌های `try...except`` قرار گرفته‌اند. این کار تضمین می‌کند که خطاهای شبکه، خطاهای API یا سایر استثنائات پیش‌بینی نشده، منجر به از کار افتادن ربات نشوند. در صورت بروز خطا، پیام مناسبی به کاربر ارسال می‌شود و جزئیات خطا در سیستم لاگینگ ثبت می‌گردد تا برای عیب‌یابی‌های بعدی قابل دسترسی باشد.
- بازنشانی وضعیت کاربر: در برخی موارد بحرانی (مانند عدم دسترسی به پایگاه داده یا خطاهای مکرر در ارسال سوالات آزمون)، وضعیت کاربر به `MAIN_MENU`` بازنشانی می‌شود. این کار از گیر افتادن کاربر در یک وضعیت نامطلوب جلوگیری می‌کند و به او اجازه می‌دهد تا از ابتدا شروع کند یا با دستور `start/`` به منوی اصلی بازگردد.

این رویکردهای جامع در مدیریت خطا، به پایداری و قابلیت اطمینان ربات کمک شایانی می‌کند و تجربه کاربری را حتی در مواجهه با شرایط غیرمنتظره، بهبود می‌بخشد.





طراحی شده‌اند که هر موجودیت (مانند کاربر، لغت، درس گرامر) دارای یک جدول مستقل باشد و اطلاعات مربوط به آن موجودیت تنها در یک مکان ذخیره شود. روابط بین جداول از طریق کلیدهای خارجی (Foreign Keys) برقرار شده‌اند که تضمین می‌کنند ارجاعات بین جداول معتبر هستند. به عنوان مثال، `user\_id` در جدول `progress` یک کلید خارجی است که به `user\_id` در جدول `users` ارجاع می‌دهد و این امر از ثبت پیشرفت برای کاربران ناموجود جلوگیری می‌کند.

### 3-4-3 استراتژی های indexing

برای بهبود عملکرد کوئری‌ها، به ویژه در جداولی که عملیات جستجو و فیلتر کردن زیادی روی آن‌ها انجام می‌شود، ایندکس‌هایی (Indexes) ایجاد شده‌اند. ایندکس‌ها ساختارهای داده‌ای هستند که سرعت بازیابی ردیف‌ها را بر اساس مقادیر یک یا چند ستون افزایش می‌دهند. در این پروژه، ایندکس‌های زیر برای بهینه‌سازی عملکرد استفاده شده‌اند:

- PRIMARY KEY: به طور خودکار بر روی ستون‌های `id` در تمام جداول ایجاد می‌شود و جستجو بر اساس کلید اصلی را بسیار سریع می‌کند.
- idx\_user\_conversation\_user\_level: این ایندکس بر روی ستون‌های `user\_id` و `level` در جدول `user\_conversation` ایجاد شده است. هدف از این ایندکس، سرعت بخشیدن به کوئری‌هایی است که موضوعات مکالمه دیده شده توسط یک کاربر خاص در یک سطح زبانی مشخص را بازیابی می‌کنند. ایندکس‌گذاری بر روی این دو ستون که اغلب در شرط `WHERE` کوئری‌ها استفاده می‌شوند، زمان پاسخگویی پایگاه داده را به طور قابل توجهی کاهش می‌دهد.

استفاده از ایندکس‌ها به ویژه در سناریوهایی که تعداد کاربران و حجم داده‌ها افزایش می‌یابد، اهمیت پیدا می‌کند. با این حال، باید توجه داشت که ایندکس‌ها سربار (Overhead) نیز دارند؛ زیرا هر بار که داده‌ای در

جدول تغییر می‌کند (درج، به‌روزرسانی، حذف)، ایندکس نیز باید به‌روزرسانی شود. بنابراین، ایندکس‌گذاری باید با دقت و تنها بر روی ستون‌هایی که به طور مکرر در عملیات جستجو و فیلتر استفاده می‌شوند، انجام شود.

### 3-5 ساختار پروژه و فایل‌ها

ساختار پروژه به گونه‌ای طراحی شده است که هر فایل و ماژول دارای مسئولیت واحد و مشخصی باشد (Single Responsibility Principle). این رویکرد، خوانایی، قابلیت نگهداری و توسعه‌پذیری کد را به شدت افزایش می‌دهد. در ادامه، به تشریح فایل‌ها و ماژول‌های کلیدی پروژه می‌پردازیم:

#### 3-5-1 فایل‌ها و نقش آن‌ها

- `bot.py`: این فایل هسته اصلی ربات است و به عنوان ارکستراتور عمل می‌کند. مسئولیت آن، راه‌اندازی ربات، ثبت هندلرها، مدیریت جریان‌های اصلی مکالمه و هماهنگی بین سایر ماژول‌ها است. این فایل منطق کسب‌وکار اصلی را در خود جای نمی‌دهد، بلکه وظایف را به ماژول‌های تخصصی‌تر واگذار می‌کند.
- `user_db.py`: این ماژول یک لایه دسترسی به داده (Data Access Layer - DAL) برای پایگاه داده `'user_data.db'` فراهم می‌کند. تمام عملیات مربوط به کاربران (مانند ثبت‌نام، به‌روزرسانی سطح، ذخیره پیشرفت) از طریق توابع تعریف شده در این ماژول انجام می‌شود. این جداسازی، هسته ربات را از جزئیات پیاده‌سازی پایگاه داده بی‌نیاز می‌کند.
- `content_manager.py`: مشابه `'user_db.py'`، این ماژول نیز یک DAL برای پایگاه داده `'content_data.db'` است. وظیفه آن، واکنشی محتوای آموزشی (لغات، گرامر، سوالات آزمون) بر اساس سطح کاربر و سایر معیارها است. این ماژول همچنین شامل منطق مربوط به Fallback برای محتوا است.

- `.env`: این فایل برای مدیریت متغیرهای محیطی حساس مانند `TELEGRAM_BOT_TOKEN` و `OPENAI_API_KEY` استفاده می‌شود. استفاده از این فایل، امنیت پروژه را افزایش می‌دهد، زیرا این اطلاعات حساس مستقیماً در کد قرار نمی‌گیرند و می‌توانند به راحتی در محیط‌های مختلف (توسعه، تست، تولید) مدیریت شوند. این فایل در `.gitignore` لیست شده است تا از آپلود تصادفی آن به مخازن عمومی جلوگیری شود.
- `requirements.txt`: این فایل تمام وابستگی‌های پایتون پروژه را لیست می‌کند. با استفاده از `pip install -r requirements.txt`، می‌توان تمام کتابخانه‌های مورد نیاز را به راحتی نصب کرد و از سازگاری محیط توسعه اطمینان حاصل نمود.
- `test_content_db.py` و `test_openai.py`: این فایل‌ها شامل تست‌های واحد (Unit Tests) برای ماژول‌های `content_manager.py` و توابع مرتبط با `OpenAI API` هستند. وجود تست‌ها به اطمینان از صحت عملکرد اجزای مختلف سیستم کمک می‌کند و فرآیند توسعه و اعمال تغییرات را ایمن‌تر می‌سازد.
- فایل‌های مستندات: `API_REFERENCE.md`، `PROJECT_DOCUMENTATION.md`، `DEPLOYMENT_GUIDE.md`، `TECHNICAL_GUIDE.md`، `USER_MANUAL.md`: این فایل‌ها مستندات مختلف پروژه را شامل می‌شوند که برای توسعه‌دهندگان، مدیران سیستم و کاربران نهایی مفید هستند. این مستندات شامل راهنمای API، راهنمای استقرار، مستندات فنی پروژه و راهنمای کاربری هستند.

## 3-5-2 نمونه کدهای کلیدی و توضیحات

در این بخش، به بررسی نمونه کدهای مهم از ماژول‌های مختلف و تشریح عملکرد آن‌ها می‌پردازیم.

## 1. `bot.py` - ثبت هندلرها و مدیریت وضعیت

همانطور که در بخش ۲-۳-۳ توضیح داده شد، `ConversationHandler` نقش محوری در مدیریت جریان مکالمه دارد. نمونه کد زیر نحوه تعریف و ثبت یک `ConversationHandler` برای آزمون تعیین سطح را نشان می‌دهد:

```
#bot.py

... #imports and other initializations

#Define conversation states

MAIN_MENU, LEVEL_ASSESSMENT, VOCABULARY_PRACTICE,
GRAMMAR_LESSON, CONVERSATION_PRACTICE, VOCABULARY_TEST =
range(6)

user_states {} =

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    ... # user registration and initial message logic

    pass

async def assess_level(update: Update, context: ContextTypes.DEFAULT_TYPE):
    ... # assessment initialization logic

    pass

async def handle_assessment_answer(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    ... # logic to process assessment answer and send next question

    pass

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    ... # logic to cancel conversation

    pass
```

```

def main:()

... # Updater and Dispatcher initialization

# Define ConversationHandler for the assessment
test_conv_handler = ConversationHandler(
    entry_points=[CommandHandler("assess_level", assess_level)],
    states={
        LEVEL_ASSESSMENT:
            MessageHandler(filters.TEXT & ~filters.COMMAND,
handle_assessment_answer),
            CallbackQueryHandler(handle_assessment_answer) # For inline
keyboard answers
    },
    fallbacks=[CommandHandler("cancel", cancel)],
    allow_reentry=True # Allows re-entering the conversation if user sends
/assess_level again
(

    dispatcher.add_handler(CommandHandler("start", start))

    dispatcher.add_handler(test_conv_handler)

... # add other handlers

... # start polling

if __name__ == '__main__':
    main()

```

در این کد، 'entry\_points' مشخص می‌کند که با دستور 'assess\_level/'، مکالمه آزمون تعیین سطح آغاز می‌شود و تابع 'assess\_level' فراخوانی می‌گردد. 'states' شامل 'LEVEL\_ASSESSMENT' است که در آن 'handle\_assessment\_answer' مسئول پردازش پاسخ‌های کاربر است. 'fallbacks' نیز برای مدیریت دستور 'cancel/' استفاده می‌شود تا کاربر بتواند از آزمون خارج شود.

## 2. 'user\_db.py' - مدیریت کاربران و پیشرفت

ماژول 'user\_db.py' شامل توابعی برای تعامل با جدول 'users' و 'progress' است. تابع 'register\_user' نمونه‌ای از نحوه مدیریت ثبت‌نام کاربران جدید و به‌روزرسانی اطلاعات آن‌ها است:

```
#user_db.py

import sqlite3

from datetime import datetime

import logging

logger = logging.getLogger(__name__)

class UserDatabase:

    def __init__(self, db_path="user_data.db"):

        self.conn = sqlite3.connect(db_path)

        self.cursor = self.conn.cursor()

        self.init_database()

    def init_database(self):

        try:

            self.cursor.execute("""

                CREATE TABLE IF NOT EXISTS users)
```

```

        user_id INTEGER PRIMARY KEY,

        username TEXT,

        level TEXT DEFAULT 'beginner,'

        join_date TEXT,

        last_active TEXT,

        assessment_done BOOLEAN DEFAULT FALSE

(
(
    self.conn.commit()

    logger.info("Users table initialized successfully.")

except sqlite3.Error as e:

    logger.error(f"Error initializing users table: {e}")

def register_user(self, user_id, username):

    try:

        self.cursor.execute("SELECT username FROM users WHERE user_id =
        ?", (user_id,))

        existing_user = self.cursor.fetchone()

        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        if existing_user:

            if existing_user[0] != username:

                self.cursor.execute(

                    "
                    UPDATE users SET username = ?, last_active = ? WHERE
                    user_id, "? =

                )

                username, now, user_id(

(

```



```

        self.conn.commit()

        logger.info(f"Updated username for user {user_id} to {username}.")
    else:

        self.cursor.execute)

    "        UPDATE users SET last_active = ? WHERE user_id, "? =
    )        now, user_id(
    (

        self.conn.commit()

        return False # User already existed
    else:

        self.cursor.execute)

    "        INSERT INTO users (user_id, username, join_date, last_active,
assessment_done) VALUES, "(?, ?, ?, ?, ?)
    )        user_id, username, now, now, 0(
    (

        self.conn.commit()

        logger.info(f"Registered new user: {user_id} ({username}).")

        return True # New user was added
except sqlite3.Error as e:

    logger.error(f"Error registering user {user_id}: {e}")

    return False

def save_assessment_result(self, user_id, percentage):

    try:

        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        level = self.get_user_level(user_id) # Get current level to save with
progress

        self.cursor.execute)

```

```

"          INSERT INTO progress (user_id, section, level, score, date) VALUES
, "(?, ?, ?, ?, ?)"

)          user_id, "assessment", level, percentage, now(

(

    self.conn.commit()

    logger.info(f"Saved assessment result for user {user_id}: {percentage}%")

    return True, None

except sqlite3.Error as e:

    error_msg = f"Error saving assessment result for user {user_id}: {e}"

    logger.error(error_msg)

    return False, error_msg

... # other methods like get_user_level, update_user_level, etc()

```

تابع `register_user` ابتدا بررسی می‌کند که آیا کاربر با `user_id` مشخص قبلاً در پایگاه داده وجود دارد یا خیر. اگر وجود داشته باشد، نام کاربری و زمان آخرین فعالیت او به‌روزرسانی می‌شود. در غیر این صورت، یک رکورد جدید برای کاربر ایجاد می‌گردد. تابع `save_assessment_result` نیز مسئول ذخیره نتیجه آزمون تعیین سطح کاربر در جدول `progress` است.

### 3. `content_manager.py` – واکشی محتوا و Fallback

ماژول `content_manager.py` مسئول ارائه محتوای آموزشی است. تابع `get_vocabulary_for_level` نمونه‌ای از نحوه واکشی لغات بر اساس سطح کاربر و مدیریت لغات مطالعه شده است:

```

``python

#content_manager.py

import sqlite3

```

```

import random

import logging

logger = logging.getLogger(__name__)

class ContentManager:

    def __init__(self, db_path="content_data.db", user_db_path="user_data.db"):

        self.conn = sqlite3.connect(db_path)

        self.cursor = self.conn.cursor()

        self.user_conn = sqlite3.connect(user_db_path) # Connection to
user_data.db

        self.user_cursor = self.user_conn.cursor()

        self.init_database()

    def init_database(self):

        try:

            self.cursor.execute("""

                CREATE TABLE IF NOT EXISTS vocabulary_words)

                id INTEGER PRIMARY KEY AUTOINCREMENT,

                word TEXT,

                definition TEXT,

                example TEXT,

                level TEXT

            (

            (""")

            self.conn.commit()

            logger.info("Vocabulary words table initialized.")

        ) ... #         create other content tables(

```

```

self.cursor.execute("SELECT COUNT(*) FROM vocabulary_words")

count = self.cursor.fetchone()[0]()

if count == 0:

    self.populate_vocabulary_table() # Populate if empty

    logger.info("Populated vocabulary words table with initial data.")


except sqlite3.Error as e:

    logger.error(f"Error initializing content database: {e}")


def get_studied_words(self, user_id):

    try:

        self.user_cursor.execute)

        "        SELECT word FROM vocabulary WHERE user_id, "? =
    )        user_id(,
    (

        return set(row[0] for row in self.user_cursor.fetchall())

    except sqlite3.Error as e:

        logger.error(f"Error getting studied words for user {user_id}: {e}")

        return set()


def get_vocabulary_for_level(self, level, count=5, user_id=None):

    try:

        studied_words = self.get_studied_words(user_id) if user_id else set()

        if studied_words:

            placeholders = ",".join("? " * len(studied_words))

```

```

        query = f"SELECT word, definition, example FROM vocabulary_words
WHERE level = ? AND word NOT IN ({placeholders}) ORDER BY RANDOM()
LIMIT"?

```

```

        params = [level] + list(studied_words) + [count]

```

```

    else:

```

```

        query = "SELECT word, definition, example FROM vocabulary_words
WHERE level = ? ORDER BY RANDOM() LIMIT"?

```

```

        params = [level, count]

```

```

    self.cursor.execute(query, params)

```

```

    words[] =

```

```

    for row in self.cursor.fetchall():

```

```

        words.append{)

```

```

        '        word': row,[0]

```

```

        '        definition': row,[1]

```

```

        '        example': row[2]

```

```

    ({

```

```

        if len(words) < count:

```

```

            logger.warning(f"Not enough words in DB for level {level}. Using
fallback.")

```

```

            fallback_words = self.get_fallback_vocabulary(level, count * 2) # Get
more to filter

```

```

            if user_id:

```

```

                fallback_words = [w for w in fallback_words if w['word'] not in
studied_words]

```

```

            needed_count = count - len(words)

```

```

            words.extend(random.sample(fallback_words, min(needed_count,
len(fallback_words))))

```

```

        return words

    except sqlite3.Error as e:

        logger.error(f"Database error getting vocabulary for level {level}: {e}")

    #         Fallback to static lists if DB fails

    fallback_words = self.get_fallback_vocabulary(level, count)

    if user_id:

        studied_words = self.get_studied_words(user_id)

        fallback_words = [w for w in fallback_words if w['word'] not in
studied_words]

        return random.sample(fallback_words, min(count, len(fallback_words)))

    def get_fallback_vocabulary(self, level, count):

    #         This method contains hardcoded vocabulary lists for each level

    ) ... #         implementation details of static vocabulary lists(

        pass

'''

```

تابع `get_vocabulary_for_level` ابتدا سعی می‌کند لغات را از پایگاه داده `vocabulary_words` واکشی کند، با این شرط که لغات قبلاً توسط کاربر مطالعه نشده باشند (با استفاده از `get_studied_words`). اگر تعداد لغات کافی در پایگاه داده یافت نشد، از تابع `get_fallback_vocabulary` استفاده می‌کند که شامل لیست‌های لغات از پیش تعریف شده است. این مکانیزم Fallback تضمین می‌کند که ربات همیشه قادر به ارائه محتوا باشد، حتی اگر پایگاه داده خالی یا دچار مشکل باشد.

#### 4. `bot.py` - تابع `validate_user_input`

تابع `validate_user_input` در `bot.py` نقش حیاتی در تضمین کیفیت ورودی‌های کاربر و جلوگیری از ارسال داده‌های نامعتبر به OpenAI API دارد. این تابع ورودی‌های متنی کاربر را از جنبه‌های مختلف اعتبارسنجی می‌کند:

```

'''python
#bot.py
)... #imports(
import re

def validate_user_input(text: str, context_word: str = None) -> tuple[bool, str]:
    """
    Validate user input for English learning exercises.
    Returns: (is_valid, reason_if_invalid)
    """

    text = text.strip()

    .1 # Check for minimum length
    if len(text) < 3:
        return False "پاسخ شما خیلی کوتاه است. لطفاً حداقل ۳ حرف وارد کنید.",

    .2 # Check for random characters or numbers only (e.g., "!@#$%")
    if re.fullmatch(r'[0-9 ,+[-`~/?<>.,;:/{}|!|=+_()*&^%$#@/text):
        return False "لطفاً از کلمات انگلیسی معنی‌دار استفاده کنید. فقط اعداد یا نمادها مجاز نیستند.",

    .3 # Check for excessive repetition of characters (e.g., "aaaaa")
    if re.search(r'(\1{4,}', text): # 5 or more same characters in a row
        return False "aaaaa", "از تکرار غیر ضروری حروف خودداری کنید. (مثال:

```

```

.4 # Check if it contains at least some English letters

if not re.search(r'[a-zA-Z]', text):

    return False, "لطفاً از حروف انگلیسی استفاده کنید."

.5 # Check for gibberish patterns (simple heuristic: valid English words usually
have vowels)

words = text.split()

valid_words_count = 0

for word in words:

    if re.search(r'[aeiouAEIOU]', word) and len(word) > 1:

        valid_words_count += 1

if len(words) > 1 and valid_words_count / len(words) < 0.5:

    return False, "لطفاً کلمات انگلیسی معنی‌دار استفاده کنید. به نظر می‌رسد ورودی شما نامفهوم است."

.6 # Check for context word if provided (for vocabulary exercises)

if context_word and context_word.lower() not in text.lower():

    return False, f"{' ' * 10}context_word" "لطفاً از کلمه '{f}' در جمله خود استفاده کنید."

return True, "" # Input is valid
'''

```

این تابع با بررسی‌های منظم (Regular Expressions) و منطق ساده، از ورود داده‌های بی‌معنی یا مخرب به سیستم جلوگیری می‌کند. این کار نه تنها تجربه کاربری را بهبود می‌بخشد، بلکه هزینه‌های مربوط به فراخوانی‌های غیرضروری OpenAI API را نیز کاهش می‌دهد.



## 3-6 مهندسی پرامپت (Prompt Engineering) و سیستم امتیازدهی

یکی از نوآورانه‌ترین جنبه‌های این پروژه، استفاده از مدل‌های زبانی بزرگ (LLMs) مانند GPT-3.5 Turbo برای ارزیابی کیفی پاسخ‌های کاربر و ارائه بازخورد هوشمند است. این رویکرد، جایگزین قوانین ثابت و از پیش تعریف شده می‌شود و امکان ارزیابی پویا و شخصی‌سازی شده را فراهم می‌آورد. موفقیت این رویکرد به شدت به کیفیت پرامپت‌های طراحی شده بستگی دارد.

### 3-6-1 طراحی پرامپت

پرامپت‌ها دستورالعمل‌هایی هستند که به LLM داده می‌شوند تا وظیفه خاصی را انجام دهد. طراحی پرامپت‌ها شامل تعیین نقش (Role)، ارائه زمینه (Context)، تعریف وظیفه (Task) و مشخص کردن فرمت خروجی (Output Format) است. در اینجا، نمونه‌ای از پرامپت استفاده شده برای ارزیابی تمرینات لغات آورده شده است:

- پرامپت ارزیابی تمرین لغات (Vocabulary Practice Evaluation Prompt):

'''

*System: You are a precise and encouraging English language examiner AI. Your task is to evaluate a student's sentence.*

*User:*

*The student was given the word 'ubiquitous' and was asked to use it in a sentence.*

*The student's sentence is: "The internet is very ubiquitous in modern life".*

*Please evaluate the sentence based on the following criteria:*

*\*\*1. Correct usage of 'ubiquitous': Does the sentence use the word correctly in context? (Weight: 50%)*

*\*\*2. Grammatical correctness: Is the sentence grammatically sound? (Weight: 30%)*

*\*\*3.Naturalness and creativity\*\*: Does the sentence sound natural? Is it a creative example? (Weight: 20%)*

*Provide a response in a strict JSON format. The JSON object must contain two keys:*

*"-score": An integer score from 0 to 100.*

*"-feedback": A single, concise sentence (less than 20 words) explaining the score. If there's a mistake, gently correct it.*

'''

#### • تحلیل پرامپت:

نقش (Role): با تعیین 'You are a precise and encouraging English language examiner AI'، به مدل دستور داده می‌شود که با چه شخصیتی و چه رویکردی (دقیق و تشویق‌کننده) به ارزیابی بپردازد. زمینه (Context): اطلاعاتی مانند کلمه داده شده به دانش‌آموز و جمله ساخته شده توسط او، به مدل ارائه می‌شود تا ارزیابی را در بستر صحیح انجام دهد. وظیفه (Task): از مدل خواسته می‌شود که جمله را بر اساس سه معیار مشخص (استفاده صحیح از کلمه، صحت گرامری، طبیعی بودن و خلاقیت) ارزیابی کند. وزن‌دهی به هر معیار، به مدل کمک می‌کند تا اولویت‌ها را در ارزیابی درک کند. فرمت خروجی (Output Format): تاکید بر 'strict JSON format' و مشخص کردن کلیدهای 'score' و 'feedback'، تضمین می‌کند که خروجی مدل قابل پردازش خودکار توسط کد پایتون باشد. این امر برای استخراج امتیاز و بازخورد و ثبت آن‌ها در سیستم بسیار حیاتی است.

این رویکرد مهندسی پرامپت، به ربات اجازه می‌دهد تا ارزیابی‌های هوشمندانه و دقیقی را انجام دهد که فراتر از تطبیق ساده با الگوهای از پیش تعریف شده است. این قابلیت، تجربه یادگیری را برای کاربر بسیار غنی‌تر و شخصی‌سازی شده‌تر می‌کند.

- مثال خروجی:

```
{
  "score": 95,
  "feedback": "Excellent use of the word, your sentence is grammatically perfect and sounds very natural!"
}
```

این پرامپت دقیق، مدل را وادار می‌کند تا یک خروجی ساختاریافته و قابل اعتماد تولید کند که مستقیماً در سیستم امتیازدهی و بازخورد به کاربر قابل استفاده است.

## 2-6-3 مکانیزم امتیازدهی

پس از دریافت پاسخ JSON از OpenAI API، امتیاز (score) به صورت مستقیم از فیلد `score` استخراج می‌شود. این امتیاز، یک عدد صحیح بین ۰ تا ۱۰۰ است که نشان‌دهنده کیفیت کلی پاسخ کاربر بر اساس معیارهای تعریف شده در پرامپت است. این امتیاز سپس در پایگاه داده `user\_data.db` و در جدول `progress` ذخیره می‌شود. این مکانیزم به ربات اجازه می‌دهد تا پیشرفت کاربر را در طول زمان ردیابی کند و بر اساس آن، محتوای آموزشی و تمرینات بعدی را تنظیم نماید.

## چالش‌ها و راه‌حل‌ها در مهندسی پرامپت:

- اطمینان از پاسخ‌های دقیق و مرتبط: گاهی اوقات LLM ها ممکن است پاسخ‌های عمومی یا نامربوط ارائه دهند. برای مقابله با این چالش، پرامپت‌ها به گونه‌ای طراحی شده‌اند که بسیار خاص و محدودکننده باشند و از مدل بخواهند که به طور مستقیم به سوالات پاسخ دهد و از حاشیه‌روی پرهیز کند. همچنین، استفاده از مثال‌های (Few-shot examples) در پرامپت می‌تواند به مدل کمک کند تا فرمت و نوع پاسخ مورد انتظار را بهتر درک کند.

- جلوگیری از سوگیری LLM : (Bias)ها ممکن است سوگیری‌های موجود در داده‌های آموزشی خود را منعکس کنند. با نظارت مداوم بر بازخوردهای تولید شده و تنظیم پرامپت‌ها، تلاش می‌شود تا بازخوردها عادلانه و بدون سوگیری باشند.
- مدیریت توکن و هزینه: مکالمات طولانی با LLMها می‌تواند پرهزینه باشد. برای بهینه‌سازی، تلاش شده است تا پرامپت‌ها تا حد امکان مختصر و مفید باشند. همچنین، در آینده می‌توان مکانیزم‌هایی مانند خلاصه‌سازی مکالمات قبلی یا استفاده از مدل‌های کوچک‌تر برای وظایف ساده‌تر را پیاده‌سازی کرد.

این سیستم امتیازدهی هوشمند، یک گام بزرگ در جهت شخصی‌سازی فرآیند یادگیری زبان است و به کاربران بازخوردی می‌دهد که نه تنها دقیق است، بلکه به آن‌ها انگیزه می‌دهد تا به یادگیری خود ادامه دهند.

### 3-7 مدیریت خطا، لاگینگ و امنیت

یک سیستم نرم‌افزاری قوی و پایدار باید قادر به مدیریت صحیح خطاها، ثبت رویدادها برای عیب‌یابی و نظارت، و محافظت از داده‌ها و عملکرد خود در برابر تهدیدات امنیتی باشد. در این پروژه، رویکردهای زیر برای اطمینان از این موارد اتخاذ شده است:

#### 3-7-1 مدیریت خطا

مدیریت خطا در این سیستم به گونه‌ای طراحی شده است که از توقف ناگهانی برنامه جلوگیری کرده و در صورت بروز مشکل، اطلاعات کافی برای عیب‌یابی را فراهم آورد. رویکردهای اصلی عبارتند از:

**مدیریت خطاهای شبکه و API:** تمام فراخوانی‌ها به سرویس‌های خارجی مانند Telegram API و OpenAI API در بلوک‌های `try...except` قرار گرفته‌اند. این کار تضمین می‌کند که خطاهایی مانند

`TimeoutError` (مشکل در ارتباط شبکه) یا `APIError` (خطای سمت سرویس دهنده) به درستی مدیریت شوند. در صورت بروز چنین خطاهایی، یک پیام خطای دوستانه به کاربر ارسال می‌شود (مثلاً "متأسفانه مشکلی در ارتباط با سرور پیش آمد. لطفاً دوباره تلاش کنید.") و جزئیات خطا در لاگ‌ها ثبت می‌گردد.

**مدیریت خطاهای پایگاه داده:** تعاملات با پایگاه داده SQLite نیز در بلوک‌های `try...except` قرار دارند. خطاهایی مانند `sqlite3.IntegrityError` (مثلاً تلاش برای درج رکورد تکراری با کلید اصلی یکسان) یا `sqlite3.OperationalError` (مشکلات عملیاتی پایگاه داده) به طور خاص مدیریت می‌شوند. این امر از فساد داده‌ها جلوگیری کرده و پایداری سیستم را افزایش می‌دهد.

**اعتبارسنجی ورودی:** همانطور که در بخش‌های قبل هم توضیح داده شد، تابع `validate\_user\_input` ورودی‌های کاربر را قبل از پردازش اعتبارسنجی می‌کند. این یک لایه دفاعی اولیه در برابر داده‌های نامعتبر یا مخرب است که می‌تواند منجر به خطاهای منطقی یا امنیتی شود.

## 3-7-2 لاگینگ (Logging)

سیستم لاگینگ با استفاده از ماژول استاندارد `logging` پایتون پیاده‌سازی شده است. این ماژول امکان ثبت رویدادها را در سطوح مختلف فراهم می‌کند که برای نظارت، عیب‌یابی و تحلیل عملکرد ربات بسیار مفید است:

**`logging.INFO`:** برای ثبت رویدادهای عادی و اطلاعاتی که نشان‌دهنده عملکرد صحیح سیستم هستند، مانند ثبت نام کاربر جدید، شروع یک آزمون، یا ارسال موفقیت‌آمیز یک پیام. این لاگ‌ها برای نظارت بر فعالیت کلی ربات استفاده می‌شوند.

**`logging.WARNING`:** برای ثبت شرایطی که غیرمنتظره هستند اما لزوماً منجر به توقف برنامه نمی‌شوند، مانند عدم یافتن محتوای کافی در پایگاه داده برای یک سطح خاص، یا دریافت یک دستور ناشناخته از کاربر.

این هشدارها به توسعه‌دهنده کمک می‌کنند تا مشکلات احتمالی را قبل از تبدیل شدن به خطاهای جدی شناسایی کند.

`logging.ERROR` برای ثبت خطاهای جدی که مانع از انجام یک عملیات خاص می‌شوند، مانند عدم موفقیت در به‌روزرسانی سطح کاربر در پایگاه داده، یا خطای ارتباط با OpenAI API. این لاگ‌ها برای عیب‌یابی فوری و رفع مشکلات بحرانی استفاده می‌شوند.

لاگ‌ها به صورت پیش‌فرض در خروجی استاندارد (کنسول) نمایش داده می‌شوند، اما می‌توانند به فایل نیز هدایت شوند تا برای تحلیل‌های بعدی در دسترس باشند. این سیستم لاگینگ جامع، ابزاری حیاتی برای حفظ سلامت و عملکرد ربات در محیط عملیاتی است.

### 3-7-3 امنیت

امنیت یکی از جنبه‌های حیاتی در توسعه هر سیستم نرم‌افزاری است، به ویژه سیستمی که با داده‌های کاربر و API‌های خارجی حساس سروکار دارد. در این پروژه، ملاحظات امنیتی زیر در نظر گرفته شده است:

#### • مدیریت توکن‌ها و کلیدها: `TELEGRAM\_BOT\_TOKEN` و `OPENAI\_API\_KEY`

اطلاعات بسیار حساسی هستند که نباید مستقیماً در کد قرار گیرند یا به مخازن عمومی (مانند GitHub) آپلود شوند. برای مدیریت این توکن‌ها، از فایل `.env` و کتابخانه `python-dotenv` استفاده شده است. این فایل در `.gitignore` لیست شده است تا اطمینان حاصل شود که هرگز به صورت تصادفی به مخزن کد اضافه نمی‌شود. این روش، امنیت اطلاعات حساس را تضمین می‌کند و امکان مدیریت آسان متغیرهای محیطی را در محیط‌های مختلف فراهم می‌آورد.

- **جلوگیری از SQL Injection:** تمام کوئری‌های پایگاه داده که شامل ورودی کاربر هستند، با استفاده از پارامترهای جایگزین (Placeholders) مانند '?' در SQLite پیاده‌سازی شده‌اند. این روش (Prepared Statements) تضمین می‌کند که ورودی کاربر به عنوان داده تفسیر شود و نه به عنوان بخشی از دستور SQL، که به طور موثر از حملات SQL Injection جلوگیری می‌کند. به عنوان مثال، به جای الحاق مستقیم رشته‌ها، از "`, `", `?" WHERE user\_id = ?" استفاده می‌شود.
- **اعتبارسنجی ورودی کاربر:** همانطور که در بخش مدیریت خطا ذکر شد، اعتبارسنجی دقیق ورودی کاربر یک لایه امنیتی مهم است. این کار از ورود داده‌های مخرب یا غیرمنتظره که می‌توانند منجر به آسیب‌پذیری‌های امنیتی (مانند حملات XSS در صورت وجود رابط وب، یا حملات منطقی) شوند، جلوگیری می‌کند.
- **Rate Limiting (پیشنهادی برای آینده):** با توجه به هزینه‌های مرتبط با OpenAI API و جلوگیری از سوءاستفاده، پیاده‌سازی مکانیزم Rate Limiting برای محدود کردن تعداد درخواست‌هایی که یک کاربر می‌تواند در یک بازه زمانی مشخص ارسال کند، در آینده ضروری خواهد بود. این کار می‌تواند با ذخیره زمان آخرین درخواست هر کاربر در پایگاه داده و بررسی آن قبل از هر فراخوانی API انجام شود.
- **مدیریت نشست‌ها (Session Management - پیشنهادی برای آینده):** در حال حاضر، وضعیت کاربر در حافظه ('user\_states' دیکشنری) نگهداری می‌شود. برای افزایش پایداری و امنیت، به ویژه در محیط‌های توزیع شده یا در صورت ریسک شدن ربات، می‌توان وضعیت نشست کاربر را در پایگاه داده یا یک سیستم کش توزیع شده (مانند Redis) ذخیره کرد. این کار همچنین به مدیریت بهتر نشست‌های فعال و جلوگیری از حملات ربایش نشست کمک می‌کند.

با پیاده‌سازی این رویکردهای مدیریت خطا، لاگینگ و امنیت، ربات هوشمند آموزش زبان به یک سیستم قابل اعتماد، پایدار و امن تبدیل می‌شود که می‌تواند با اطمینان در محیط عملیاتی به کاربران خدمات ارائه دهد.



## فصل 4

### نتایج و ارزیابی

#### 1-4 مقدمه

این فصل به جمع‌بندی نتایج حاصل از طراحی و پیاده‌سازی “ربات هوشمند آموزش زبان” اختصاص دارد. ابتدا، دستاوردهای کلیدی پروژه و نوآوری‌های آن به صورت خلاصه مرور می‌شود. سپس، محدودیت‌های فنی و عملیاتی که در نسخه فعلی سیستم وجود دارد، به صورت شفاف مورد بحث قرار می‌گیرد. بخش اصلی این فصل به ارائه پیشنهادات مشخص برای کارهای آینده اختصاص یافته است؛ این پیشنهادات نه تنها مسیرهای پژوهشی، بلکه گام‌های عملی برای تبدیل این پروژه به یک محصول کامل و قابل استفاده در دنیای واقعی را نیز ترسیم می‌کنند. در این راستا، به اهمیت توسعه یک پنل مدیریت، سیستم آنالیز داده‌ها و ایجاد مستندات راه‌اندازی به تفصیل پرداخته خواهد شد. در نهایت، یک جمع‌بندی کلی، اهمیت این پروژه را در حوزه یادگیری زبان به کمک هوش مصنوعی برجسته می‌سازد.

## 2-4 خلاصه دستاوردها

پروژه حاضر با موفقیت یک سیستم یکپارچه آموزش زبان را بر بستر تلگرام پیاده‌سازی نمود که توانست شکاف میان ابزارهای تمرین گسسته و پلتفرم‌های مکالمه آزاد را پر کند. دستاوردهای اصلی این پروژه را می‌توان در موارد زیر خلاصه کرد:

- **ایجاد یک چرخه یادگیری کامل:** این ربات اولین ربات تلگرامی است که چرخه کاملی از تعیین سطح (Placement Test)، آموزش محتوا (Vocabulary & Grammar Lessons)، تمرین تعاملی (Interactive Practice) و مکالمه آزاد (AI Chat) را در یک محیط واحد و منسجم ارائه می‌دهد.
- **پیاده‌سازی سیستم ارزیابی هوشمند:** مهم‌ترین نوآوری پروژه، استفاده از مهندسی پرامپت پیشرفته برای بهره‌گیری از مدل GPT-3.5-turbo به عنوان یک **ارزیاب هوشمند** است. این سیستم قادر است جملات کاربران را بر اساس معیارهای چندگانه (صحت گرامری، استفاده صحیح از کلمه، طبیعی بودن) نمره‌دهی کرده و بازخورد فوری و سازنده تولید کند.
- **معماری ماژولار و قابل توسعه:** با تفکیک منطق برنامه، مدیریت پایگاه داده کاربران و مدیریت محتوای آموزشی در فایل‌های مجزا (bot.py, user\_db.py, content\_manager.py)، یک معماری تمیز و قابل نگهداری ایجاد شد که افزودن ویژگی‌های جدید در آینده را به شدت تسهیل می‌کند.
- **استفاده از پایگاه داده دوگانه:** جداسازی داده‌های پویای کاربران از محتوای ایستای آموزشی در دو پایگاه داده مجزا، به بهینه‌سازی عملکرد، سادگی مدیریت و تهیه نسخه پشتیبان کمک شایانی کرده است.

### 3-4 محدودیت‌ها

با وجود دستاوردهای ذکر شده، پروژه در نسخه فعلی با محدودیت‌هایی روبرو است که شناخت آن‌ها برای توسعه‌های آتی ضروری است:

- **هزینه عملیاتی API:** بزرگترین چالش عملیاتی پروژه، هزینه مستقیم استفاده از API شرکت OpenAI است. با افزایش تعداد کاربران و حجم مکالمات، این هزینه می‌تواند به صورت تصاعدی افزایش یافته و پایداری بلندمدت پروژه را تهدید کند.
- **وابستگی به سرویس‌های شخص ثالث:** عملکرد کل سیستم به پایداری و در دسترس بودن API‌های تلگرام و OpenAI گره خورده است. هرگونه قطعی یا تغییر در سیاست‌های این سرویس‌ها می‌تواند مستقیماً بر روی ربات تأثیر بگذارد.
- **عدم وجود رابط مدیریت گرافیکی (Admin Panel):** در حال حاضر، تمام مدیریت کاربران و محتوا نیازمند دسترسی مستقیم به پایگاه داده و اجرای کوئری‌های SQL است. این موضوع، مدیریت و نظارت بر سیستم را برای یک ادمین غیرفنی غیرممکن و برای توسعه‌دهنده، زمان‌بر و مستعد خطا می‌سازد.
- **محدودیت به تعاملات متنی:** سیستم فعلی تنها از مهارت‌های خواندن و نوشتن پشتیبانی می‌کند و فاقد هرگونه قابلیت برای تمرین مهارت‌های حیاتی شنیداری (Listening) و گفتاری (Speaking) است.
- **مقیاس‌پذیری پایگاه داده:** استفاده از SQLite برای سادگی و سرعت در توسعه ایده‌آل است، اما در مواجهه با تعداد بالای درخواست‌های همزمان (High Concurrency) که در یک ربات تلگرام محبوب محتمل است، می‌تواند دچار قفل شدن (database lock) و افت عملکرد شود.

#### 4-4 پیشنهادات برای پژوهش‌های بعدی

بر اساس محدودیت‌های شناسایی شده و چشم‌انداز پروژه، چندین مسیر مشخص برای توسعه و بهبود سیستم پیشنهاد می‌شود:

##### 4-4-1 توسعه پنل مدیریت و داشبورد تحلیلی (Admin Panel & Analytics)

این مهم‌ترین گام برای تبدیل پروژه از یک نمونه اولیه به یک سرویس قابل مدیریت است. پنل مدیریت که می‌تواند با استفاده از فریم‌ورک‌های وب پایتون مانند Flask یا Django توسعه یابد، باید شامل قابلیت‌های زیر باشد:

- **مدیریت کاربران:** مشاهده لیست کاربران، جستجو بر اساس `user_id`، ویرایش دستی سطح زبان کاربر و مشاهده سوابق پیشرفت آن‌ها.
- **مدیریت محتوا:** رابط کاربری گرافیکی برای افزودن، ویرایش و حذف کلمات، دروس گرامر و سوالات آزمون از پایگاه داده `content_data.db` بدون نیاز به دخالت مستقیم در دیتابیس.
- **داشبورد آنالیتیکس:** نمایش آمار و معیارهای کلیدی عملکرد (KPIs) به صورت نمودارهای گرافیکی برای درک بهتر رفتار کاربران و سلامت سیستم، از جمله:
- **آمار کاربران:** تعداد کاربران جدید روزانه، کاربران فعال ماهانه (MAU)، نرخ بازگشت (Retention Rate)
- **آمار یادگیری:** میانگین نمرات در سطوح مختلف، شناسایی کلمات یا مفاهیم گرامری که بیشترین چالش را برای کاربران ایجاد کرده‌اند.
- **آمار عملکرد سیستم:** میانگین زمان پاسخ API، تعداد خطاهای ثبت شده.

- **ارسال پیام همگانی (Broadcast):** قابلیت ارسال پیام‌های اطلاع‌رسانی یا آموزشی به تمام کاربران یا گروهی از آن‌ها (مثلاً فقط کاربران سطح Intermediate)

#### 4-4-2 ایجاد راهنمای جامع راه‌اندازی و استقرار (Deployment Guide)

برای اینکه پروژه قابلیت بازتولید (Reproducibility) داشته باشد و دیگران بتوانند به راحتی آن را اجرا کرده یا در توسعه آن مشارکت کنند، ایجاد مستندات دقیق راه‌اندازی ضروری است. این راهنما باید در فایلی مانند `DEPLOYMENT.md` قرار گرفته و شامل مراحل زیر باشد:

**استقرار در سرور (Deployment):** ارائه پیشنهاداتی برای استقرار پایدار ربات روی سرویس

های ابری مانند Heroku یا یک سرور مجازی (VPS) با استفاده از ابزارهایی مانند Gunicorn و Supervisor اشاره به **کانتینر سازی با Docker** به عنوان یک روش مدرن و استاندارد برای تضمین اجرای یکسان در محیط‌های مختلف.

#### 4-4-3 افزودن قابلیت‌های صوتی و چندرسانه‌ای

برای پوشش دادن مهارت‌های شنیداری و گفتاری، می‌توان از API های تشخیص گفتار (Speech-to-Text) و سنتز گفتار (Text-to-Speech) استفاده کرد. این قابلیت به کاربر اجازه می‌دهد به جای تایپ کردن، پاسخ خود را به صورت صوتی ارسال کند و ربات نیز تلفظ او را ارزیابی کرده و پاسخ‌ها را به صورت صوتی برای او پخش کند.

#### 4-4-4 پیاده‌سازی گیمیفیکیشن و مسیر یادگیری شخصی‌سازی شده

برای افزایش انگیزه کاربران، می‌توان عناصر بازی‌وارسازی (Gamification) مانند امتیاز تجربه (XP)، دستاوردها (Achievements)، رتبه‌بندی هفتگی (Leaderboard) و دنباله روزانه (Daily Streak) را به سیستم افزود. این داده‌ها همچنین می‌توانند برای ایجاد یک مسیر یادگیری شخصی‌سازی شده

**(Personalized Learning Path)** استفاده شوند که در آن، سیستم به طور هوشمند تمرین‌ها و دروسی را پیشنهاد می‌دهد که دقیقاً با نقاط ضعف کاربر مطابقت دارد.

#### 4-4-5 مهاجرت به مدل‌های متن‌باز و بهینه‌سازی (Fine-Tuning)

برای حل مشکل هزینه و وابستگی، می‌توان به سمت استفاده از مدل‌های زبانی بزرگ متن‌باز Open-source LLMs مانند سری Llama یا Mistral حرکت کرد. این مدل‌ها را می‌توان بر روی سرور شخصی میزبانی کرده و با استفاده از تکنیک **Fine-tuning** بر روی داده‌های تخصصی آموزش زبان، آن‌ها را برای وظیفه ارزیابی و مکالمه آموزشی بهینه‌سازی نمود. این کار ضمن کاهش چشمگیر هزینه‌ها، کنترل بیشتری بر عملکرد و خروجی مدل فراهم می‌کند.

#### 4-5 نتیجه‌گیری

پروژه "ربات هوشمند آموزش زبان" با موفقیت نشان داد که چگونه می‌توان با ادغام هوشمندانه پلتفرم‌های پیام‌رسان، پایگاه‌های داده و مدل‌های زبانی بزرگ، ابزارهای آموزشی قدرتمند، دسترس‌پذیر و تعاملی خلق کرد. این سیستم فراتر از یک اثبات مفهوم (Proof-of-Concept) عمل کرده و یک بنیاد مستحکم و معماری قابل اتکا برای ساخت یک محصول آموزشی کامل را فراهم آورده است. دستاوردهای این پروژه در زمینه ارزیابی هوشمند و ایجاد یک چرخه یادگیری یکپارچه، گامی مهم در جهت تحقق رویای داشتن یک مربی زبان شخصی برای همگان است. با پیاده‌سازی پیشنهادات ارائه‌شده، به خصوص توسعه پنل مدیریت و افزودن قابلیت‌های چندرسانه‌ای، این پروژه پتانسیل تبدیل شدن به یک سرویس پیشرو و تأثیرگذار در حوزه فناوری‌های آموزشی را داراست.

1. Mandalika, D., Ramadhanty, F., & Dasuki, D. (2023). Telegram bot as a digital learning platform in English language teaching and learning.
2. Freidkin, I., & Ksenofontova, S. (2021). Foreign language teaching methods of nowadays: telegram-bot “Pardes”.
3. Pikhart, M. (2020). Intelligent information processing for language education: The use of artificial intelligence in language learning apps
4. Kim, N. Y., Cha, Y., & Kim, H. S. (2019). Future English learning: Chatbots and artificial intelligence
5. Son, J. B., Ružić, N. K., & Philpott, A. (2025). Artificial intelligence technologies and applications for language learning and teaching.
6. Okonkwo, C. W., & Ade-Ibijola, A. (2021). Chatbots applications in education: A systematic review.
7. Cunningham-Nelson, S., & Boles, W. (2019). A review of chatbots in education: practical steps forward.

# واژه نامه انگلیسی به فارسی

|                                          |                                     |
|------------------------------------------|-------------------------------------|
| A                                        |                                     |
| Abstraction Layer                        | لایه انتزاع                         |
| API (Application Programming Interface)  | رابط برنامه‌نویسی کاربردی           |
| C                                        |                                     |
| Content Management                       | لایه مدیریت محتوا                   |
| ConversationHandler                      | کنترل‌کننده مکالمه                  |
| CRUD (Create, Read, Update, Delete)      | ایجاد، خواندن، به‌روزرسانی، حذف     |
| E                                        |                                     |
| ER Diagram (Entity-Relationship Diagram) | نمودار رابطه-موجودیت                |
| G                                        |                                     |
| Gamification                             | بازی‌وار سازی                       |
| GPT (Generative Pre-trained Transformer) | ترنسفورمر تولیدگر از پیش آموزش‌دیده |
| CRUD (Create, Read, Update, Delete)      | ایجاد، خواندن، به‌روزرسانی، حذف     |
| N                                        |                                     |
| Natural Language Processing (NLP)        | پردازش زبان طبیعی                   |
| GPT (Generative Pre-trained Transformer) | ترنسفورمر تولیدگر از پیش آموزش‌دیده |
| CRUD (Create, Read, Update, Delete)      | ایجاد، خواندن، به‌روزرسانی، حذف     |



## **Abstract**

This thesis deals with the design and implementation of an intelligent Telegram bot for teaching English using the capabilities of OpenAI language models such as (GPT-3.5 Turbo). The main goal of this project is to facilitate the language learning process by creating dynamic interaction with users and providing various training exercises in conversation, grammar, and vocabulary. By connecting to the OpenAI API, the robot is able to generate questions, analyze answers, automatically correct errors, and simulate conversations. Also, this bot can generate daily exercises based on the user's level. The basic structure of the bot is implemented using the python-telegram-bot library and a SQLite database of files is used to store user information, their progress, and exercises. The results of this project demonstrate the high potential of AI models in creating interactive and personalized language learning environments.

**Keywords:** Telegram bot, English language learning, Artificial intelligence, OpenAI, Natural language processing



Shahid Rajaee Teacher Training University  
Faculty of Computer Engineering  
Department of Software  
B. Sc. Thesis

Title:

**Design and implementation of a Telegram  
bot for language teaching using an artificial  
intelligence model (GPT)**

Supervisor:

Dr. Zeinab Torabi

By:

Ali Kazemzadeh

Summer 2025